Manual técnico — Sistema La Tradición Universidad Mariano Gálvez de Guatemala · Ingeniería en Sistemas · Entrega 4 · 19 de octubre de 2026

1. Descripción general del sistema

1 Propósito y alcance

La Tradición es una aplicación web de gestión para una empresa que fabrica y comercializa productos derivados de harina, principalmente tortillas y nachos. Centraliza productos, materia prima, proveedores, clientes, empleados, gastos operativos, compras, ventas y producción. Incluye autenticación y acceso diferenciado por perfiles.

2 Objetivo general

Facilitar el registro, la consulta y el control de operaciones comerciales y productivas mediante una aplicación web conectada a una base de datos relacional MySQL, aplicando integridad, validación y control de acceso.

3 Tecnologías

Tecnología	Uso
Node.js >=18	Ejecución del servidor
Express 4	Rutas y solicitudes HTTP
EJS	Plantillas de interfaz
MySQL	Almacenamiento relacional
mysql2	Acceso a MySQL
JavaScript, HTML y CSS	Lógica e interfaz
Dotenv	Configuración por entorno
express-session	Sesiones
Bcryptjs	Operaciones de hash de contraseñas
Nodemon	Desarrollo

4 Requisitos de ejecución

Se requiere Node.js 18 o superior, npm, MySQL, navegador moderno, esquema bd_tradicion y variables de entorno válidas. Los requisitos de hardware dependen del alojamiento y no se establecen como mínimos verificados en el código.

2. Arquitectura MVC y organización del proyecto

La aplicación sigue una organización Modelo–Vista–Controlador. Las rutas reciben peticiones; los middleware comprueban autenticación/autorización; los controladores validan y coordinan; los modelos ejecutan SQL; las vistas EJS presentan los resultados.

Elemento	Responsabilidad
web/app.js	Inicialización de Express, sesiones, rutas y servidor
web/config/	Configuración de variables y conexión
web/routes/	Definición de endpoints
web/controllers/	Procesamiento de solicitudes
web/models/	Consultas y operaciones sobre MySQL
web/middleware/	Controles de acceso
web/views/	Plantillas EJS
web/public/	Recursos estáticos

Flujo típico: navegador → ruta Express → middleware → controlador → modelo → MySQL → controlador → vista/respuesta.

3. Modelo relacional

El modelo relacional representa entidades principales y tablas intermedias para resolver relaciones muchos a muchos. El diagrama fue generado mediante ingeniería inversa en MySQL Workbench. Se adjunta como referencia editable .mwb y como imagen .png.

Las principales entidades de negocio son producto, materia_prima, proveedor, cliente, empleado, compras, venta, produccion, gastos_operativos y usuarios. Gastos_operativos tiene clave primaria propia y no requiere una relación obligatoria con compras, ventas o producción.

4. Tablas, relaciones y restricciones

Grupo	Tablas
Principales	producto, materia_prima, proveedor, cliente, empleado, compras, venta, produccion, gastos_operativos, usuarios
Detalles y relaciones	producto_venta, materia_prima_producto, materia_prima_produccion, detalle_compra_materia_prima, detalle_fabricacion, produccion_empleado, venta_materia_prima, detalle_compra
Secuencias	secuencia_compras, secuencia_produccion, secuencia_venta

Relación	Tabla intermedia o referencia
Proveedor → Compras	compras.id_proveedor
Cliente → Venta	venta.id_cliente
Empleado → Venta	venta.id_empleado
Venta ↔ Producto	producto_venta
Producto ↔ Materia prima	materia_prima_producto
Producción ↔ Empleado	produccion_empleado

Compra ↔ Materia prima	detalle_compra_materia_prima
Producción ↔ Producto	detalle_fabricacion

Las claves primarias identifican registros y las foráneas garantizan referencias entre tablas. La base incluye restricciones CHECK, UNIQUE y NOT NULL; la verificación final debe contabilizar restricciones explícitas del DDL para justificar el mínimo académico de 15.

5. Configuración y conexión a MySQL

Archivo	Uso
package.json	Dependencias y scripts npm
.env.example	Plantilla de variables sin secretos
.env	Valores privados, no publicar
web/config/envConfig.js	Lectura de configuración
web/config/database.js	Pool y consultas MySQL
web/app.js	Servidor Express

```
PORT=3000
NODE_ENV=development
DB_HOST=localhost
DB_PORT=3306
DB_NAME=bd_tradicion
DB_USER=usuario_aplicacion
DB_PASSWORD=REEMPLAZAR
SESSION_SECRET=REEMPLAZAR_POR_SECRETO_SEGURO
```

El ejemplo es ilustrativo: las cuentas y permisos deben configurarse en el servidor. La conexión utiliza mysql2/promise y un pool; la aplicación debe validar credenciales y conectividad antes de atender solicitudes. No publicar .env ni credenciales de prueba.

```
npm install
npm start
# Desarrollo: npm run dev
```

La aplicación dispone de una ruta /health para comprobar la respuesta HTTP. El código local revisado escuchaba en 127.0.0.1. Posteriormente se verificó la operación del servicio Node.js en Debian y la conexión remota privada a la base de datos mediante Tailscale y SSH. La publicación abierta en internet requiere una configuración y revisión de seguridad independientes.

6. Rutas, controladores y modelos

Cada uno de los nueve módulos principales dispone de controlador, modelo y rutas. La tabla presenta la correspondencia de archivos verificada en MYSQL020; las funciones exactas pueden consultarse en el código y no deben deducirse de la interfaz.

Módulo	Controlador	Modelo
producto	productoController.js	productoModel.js
materiaPrima	materiaPrimaController.js	materiaPrimaModel.js

proveedor	proveedorController.js	proveedorModel.js
cliente	clienteController.js	clienteModel.js
empleado	empleadoController.js	empleadoModel.js
gastoOperativo	gastoOperativoController.js	gastoOperativoModel.js
compra	compraController.js	compraModel.js
venta	ventaController.js	ventaModel.js
produccion	produccionController.js	produccionModel.js

El acceso y autenticación se implementan adicionalmente con authController.js, authRoutes.js y userModel.js; la página inicial usa homeController.js y homeRoutes.js.

Modelo	Funciones detectadas (lectura orientativa)
productoModel.js	obtenerTodos, obtenerPorId, crear, actualizar, eliminar
materiaPrimaModel.js	obtenerTodas, obtenerPorId, crear, actualizar, eliminar
proveedorModel.js	obtenerTodos, obtenerPorId, crear, actualizar, eliminar
clienteModel.js	obtenerTodos, obtenerPorId, crear, actualizar, eliminar
empleadoModel.js	obtenerTodos, obtenerPorId, crear, actualizar, desactivar
gastoOperativoModel.js	listar, if, porId, crear, actualizar, eliminar
compraModel.js	listar, proveedores, materias, detalle, registrar, anular
ventaModel.js	obtenerTodas, if, obtenerClientes, obtenerEmpleados, obtenerProductos, registrar, anular
produccionModel.js	obtenerTodas, obtenerProductos, obtenerRecetasDisponibles, registrarMultiple, obtenerReceta, registrar, obtenerDetalle, anular

Los módulos de catálogo incluyen altas, consultas, ediciones y eliminación o desactivación. Compras, ventas y producción permiten registrar, consultar y anular operaciones; las anulaciones preservan historial y pueden revertir inventario conforme a procedimientos y triggers.

7. Vistas, triggers, procedimientos y reglas de negocio

7.1 Vistas

Vista	Uso
vw_inventario_productos	Existencias y clasificación SIN STOCK / STOCK BAJO / DISPONIBLE
vw_ventas_detalladas	Detalle de ventas con cliente, empleado, producto, cantidad, precio y subtotal

La vista de ventas no filtra automáticamente las ventas anuladas en la definición exportada. Para reportes de ventas vigentes se debe aplicar el criterio de estado correspondiente.

7.2 Triggers

Trigger	Evento y efecto
trg_compra_aumentar_materia_prima	AFTER INSERT en detalle_compra_materia_prima: incrementa insumos
trg_validar_stock_venta	BEFORE INSERT en producto_venta: valida existencia y cantidad
trg_descontar_stock_venta	AFTER INSERT en producto_venta: descuenta productos

7.3 Procedimientos almacenados

Procedimiento	Propósito
sp_anular_compra	Anula compra y revierte existencias de insumos si es posible.
sp_anular_produccion	Revierte fabricación con comprobaciones de inventario.
sp_anular_venta	Anula venta y reintegra unidades de productos.
sp_generar_ventas_prueba	Genera ventas ficticias para pruebas; no usar en datos reales.
sp_registrar_compra_multiple	Compra con múltiples insumos desde JSON; actualiza detalles e inventario mediante trigger.
sp_registrar_produccion	Fabrica un producto con receta y consumo de materia prima.
sp_registrar_produccion_multiple	Fabrica múltiples productos y registra sus consumos.
sp_registrar_venta	Registro de venta de un producto, apoyado en el procedimiento múltiple.
sp_registrar_venta_multiple	Venta con varios productos y controles de existencias.

Las reglas incluyen impedir ventas sin existencias, validar cantidades positivas, controlar recetas y consumos, registrar detalles y motivos de anulación, conservar historial y usar transacciones/ROLLBACK en operaciones de varias tablas. Los objetos del dump original tienen DEFINER root@localhost; debe verificarse cómo quedaron los definidores y privilegios después de importarlos a MariaDB en Debian.

8. Seguridad, roles y control de acceso

8.1 Seguridad de la aplicación web

Perfil web	Módulos autorizados
Administrador	Nueve módulos principales
Producción	Productos, materia prima y producción
Ventas	Productos, clientes y ventas

La autenticación usa la tabla usuarios, sesiones y middleware de autorización. La ocultación de opciones del menú no sustituye el bloqueo de rutas del servidor; las pruebas del equipo incluyeron acceso directo a URLs restringidas.

8.2 Roles de MySQL verificados con SHOW GRANTS

Rol	Permisos constatados
rol_administrador@%	ALL PRIVILEGES sobre bd_tradicion.*
rol_produccion@%	SELECT/INSERT/UPDATE: produccion, detalle_fabricacion, materia_prima_produccion, produccion_empleado; SELECT/UPDATE: materia_prima; SELECT: producto
rol_ventas@%	SELECT/INSERT/UPDATE: cliente, venta, producto_venta; SELECT: producto

Las capturas de SHOW GRANTS demuestran que existen cuentas con nombres de rol y privilegios diferenciados. La consulta a mysql.role_edges no devolvió asignaciones a otras cuentas. Por ello, NO está demostrada la aplicación efectiva de esos roles a las conexiones MySQL del sistema.

```
SHOW GRANTS FOR 'rol_administrador'@'%';
SHOW GRANTS FOR 'rol_produccion'@'%';
SHOW GRANTS FOR 'rol_ventas'@'%';
```

8.3 Seguridad pendiente antes de publicar

- Eliminar credenciales de prueba visibles en la pantalla de inicio de sesión.
- Sustituir contraseñas de ejemplo por marcadores y excluir .env del repositorio.
- Evitar usar root como usuario habitual de conexión.
- Revisar DEFINER root@localhost de vistas, triggers y procedimientos.
- Si se decide publicar el sistema, configurar HTTPS, sesiones seguras y permisos efectivos. El acceso actual mediante Tailscale es privado y no implica exposición pública.
- Guardar capturas de SHOW GRANTS y pruebas de acceso sin revelar secretos.

9. Observaciones y comprobaciones pendientes

- Comparar el diagrama .mwb con el dump más reciente para reconciliar el número de tablas.
- Contabilizar las restricciones explícitas y documentar las 15 mínimas exigidas.
- Confirmar los datos de prueba mínimos por tabla: algunos módulos mostraron menos de 50 registros.
- Realizar instalación desde cero y validar que los scripts SQL restauren vistas, triggers y procedimientos.
- Documentar la prueba remota privada ya realizada; completar una publicación pública segura solo si forma parte de los requisitos. Revisar las credenciales de demostración antes de exponer el servicio.
- Revisar que las rutas y nombres de funciones del manual coincidan con la versión final tras integrar la corrección A6.

Anexo A. Fuentes técnicas utilizadas

- Archivo de proyecto: base_prueba_MYSQL020.zip (código y package.json).
- Exportación MySQL: Dump20261019.sql (estructura y objetos SQL).
- Diagrama editable: diagramaWorkbench.mwb y diagramaWorkbench.png.
- Resultados de SHOW GRANTS de rol_administrador, rol_produccion y rol_ventas, aportados en la conversación.
- Pruebas de módulos y permisos reportadas por el equipo en la fase del Desarrollador 1.

Actualización documental — pruebas remotas privadas (22/10/2026)

Se verificó una conexión remota privada a un servidor Toshiba con Debian 13, Node.js 20.19.2 y MariaDB 11.8.6. Desde Windows se accedió a bd_tradicion mediante MySQL Workbench, Tailscale y SSH. También se reportó el

registro de una venta desde la aplicación fuera de la red Wi-Fi local y su comprobación, junto con movimientos de inventario, en la base de datos centralizada. Este resultado no equivale a despliegue web público ni a certificación de una instalación desde cero. Conservar las evidencias y registrar fecha y responsable de la prueba.